

Universidad de los Andes

Departamento de Ingeniería de Sistemas y Computación

ISIS4826 – Robótica Móvil

Informe Proyecto #2 – 2026-1

Planificación Computacional de Caminos Geométricos,
Ejecución (Modo Autónomo) y Relocalización

Estudiante A: Yohan Felipe Gaitan

Estudiante B: Daniel Camilo Quimbay

Profesor: Fernando De la Rosa

Fecha: Abril 2026

Contents

1	Método de Planificación	2
1.1	Descripción del método computacional	2
1.2	Resultado del método de planificación	3
1.3	Traducción del resultado en comandos para el robot	3
1.4	Resultados por escena – tabla q_f vs $q_{f\text{-est}}$	3
1.5	Posibles mejoras del método de planificación	4
1.6	Decisiones de diseño y trade-offs	5
2	Método de Relocalización	5
2.1	Descripción del método	5
2.2	Análisis de errores de localización	5
2.3	Resultados – tabla $q_{f\text{-est}}$ vs q_{act}	6
2.4	Limitaciones del método de relocalización actual	7
3	Videos Ilustrativos	7

1. Método de Planificación

1.1 Descripción del método computacional

El método implementado combina la **construcción del espacio de configuración (C-Space)** mediante la suma de Minkowski y el algoritmo de búsqueda de caminos **Dijkstra** sobre una cuadrícula superpuesta al espacio de trabajo. El proceso completo se desarrolla en tres etapas principales más una etapa de justificación de parámetros.

Etapa 1 – Expansión de obstáculos (C-obstáculos)

El robot Hiwonder MentorPi tiene dimensiones reales de 18×21 cm. Dado que el robot puede rotar sobre su propio centro, se inscribe su geometría en un cuadrado de lado 30 cm, de modo que en cualquier orientación el robot permanece completamente dentro de ese cuadrado. Cada obstáculo rectangular del escenario se expande aplicando la suma de Minkowski $B \oplus (-A)$, donde B son los vértices del obstáculo y A los vértices del cuadrado del robot centrado en el origen. El resultado es un C-obstáculo convexo que representa la región donde el centro del robot no puede ubicarse sin colisionar.

Adicionalmente, los cuatro bordes del escenario se tratan como franjas de 15 cm de ancho (**ROBOT_HALF**) que también se marcan como C-obstáculos, garantizando que el robot no pueda salir del área delimitada.

Etapa 1.5 – Justificación de parámetros críticos

Durante el desarrollo se identificó que la resolución inicial de cuadrícula (**CELL_SIZE** = 30 cm, igual al robot inscrito) resultaba insuficiente para encontrar soluciones en ciertos escenarios con alta densidad de obstáculos. Específicamente, en el **Escenario 4** con múltiples obstáculos estratégicamente posicionados, Dijkstra no lograba encontrar un camino viable. Al reducir la resolución a 20 cm, el algoritmo encontró soluciones con facilidad, aunque con incremento marginal en tiempo de cómputo (típicamente < 100 ms adicionales). Esta configuración se mantiene como parámetro fijo en **CONSTANTS.py** (**CELL_SIZE** = 0.20 m).

Etapa 2 – Clasificación de la cuadrícula

Sobre el espacio de trabajo se superpone una cuadrícula con resolución de celda de **20×20 cm** (más fina que el tamaño del robot de 30×30 cm). Esta resolución más fina fue necesaria para garantizar que el algoritmo Dijkstra encuentre solución en escenarios complejos como el Escenario 4, donde la malla de 30 cm no permitía hallar caminos viables. La resolución menor requiere más tiempo de cómputo pero proporciona mayor flexibilidad en navegación. Cada celda se clasifica según la relación de sus cinco puntos de muestra (cuatro esquinas y centro) con los C-obstáculos:

- **Libre:** ningún punto de muestra cae dentro de ningún C-obstáculo.
- **Semi-libre:** al menos un punto, pero no todos, caen dentro de un C-obstáculo.
- **Ocupada:** todos los puntos de muestra caen dentro de algún C-obstáculo.

Etapa 3 – Búsqueda de camino con Dijkstra

Sobre la cuadrícula clasificada se ejecuta el algoritmo de Dijkstra con conectividad de 8 vecinos (movimientos cardinales y diagonales). El coste de cada movimiento es 1.0 para desplazamientos cardinales y $\sqrt{2}$ para diagonales. Las celdas semi-libres se incluyen en la búsqueda pero con un factor de penalización de 2.0, de modo que el algoritmo las evita si existe una alternativa completamente libre. Las celdas ocupadas se excluyen completamente. El algoritmo garantiza el camino de menor coste acumulado entre la celda de inicio y la celda de destino.

1.2 Resultado del método de planificación

El resultado del planificador es una lista ordenada de celdas (col, row) que forman el camino óptimo en la cuadrícula. Esta discretización a 0.20 m permite que Dijkstra explore más opciones de ruta, siendo particularmente crítica en escenarios con obstáculos densamente distribuidos. Esta lista se convierte a coordenadas del mundo real usando el centro de cada celda, generando una secuencia de waypoints (x, y) . La distancia total recorrida varía entre 4.59 m y 5.90 m dependiendo del escenario y la distribución de obstáculos.

1.3 Traducción del resultado en comandos para el robot

La secuencia de waypoints se transforma en una cadena de configuraciones $\langle q_0, q_{\text{rot}_{0 \rightarrow 1}}, q_1, q_{\text{rot}_{1 \rightarrow 2}}, q_2, \dots, q_n \rangle$ donde cada elemento es una tupla (x, y, θ) en metros y grados. Esta transformación sigue la siguiente lógica:

- Para avanzar del waypoint i al waypoint $i+1$, se calcula el ángulo $\alpha = \text{atan2}(\Delta y, \Delta x)$ que apunta en la dirección del siguiente punto.
- Si α difiere de la orientación actual en más de 0.5, se inserta una configuración de rotación $q_{\text{rot}_{i \rightarrow i+1}}$ con la misma posición (x, y) pero con $\theta = \alpha$. Esto indica al robot que debe girar en el lugar antes de trasladarse.
- A continuación se inserta la configuración de traslación $q_{i+1} = (x_{i+1}, y_{i+1}, \alpha)$, indicando que el robot avanza hacia adelante hasta el siguiente waypoint.
- Al final se inserta q_{rot_n} con la orientación final θ_f del problema, para que el robot quede alineado según lo especificado.

La secuencia completa se exporta a un archivo `.txt` donde cada línea contiene una configuración (x, y, θ) . El nodo ROS2 lee este archivo secuencialmente y publica los comandos de velocidad necesarios para ejecutar cada movimiento de rotación o traslación.

1.4 Resultados por escena – tabla q_f vs $q_{f\text{-est}}$

La siguiente tabla compara la configuración final teórica q_f con la configuración estimada por el robot $q_{f\text{-est}}$ al terminar la ejecución. En todos los escenarios el robot alcanzó exactamente la posición objetivo dentro de la precisión del simulador Gazebo.

Nota: La precisión perfecta (diferencia = 0.000 m, 0.00°) entre q_f y $q_{f\text{-est}}$ se debe a que el simulador

Escena	$q_f = (x, y, \theta)$	$q_{f\text{-est}} = (x, y, \theta)$	Dif. posición (m)	Dif. angular (°)
Escena 1	(3.25, 4.25, 90°)	(3.25, 4.25, 90°)	0.000	0.00
Escena 2	(3.25, 4.25, 90°)	(3.25, 4.25, 90°)	0.000	0.00
Escena 3	(3.25, 4.25, 90°)	(3.25, 4.25, 90°)	0.000	0.00
Escena 4	(3.25, 4.25, 90°)	(3.25, 4.25, 90°)	0.000	0.00
Escena 5	(3.25, 4.25, 90°)	(3.25, 4.25, 90°)	0.000	0.00
Escena 6	(3.25, 4.25, 90°)	(3.25, 4.25, 90°)	0.000	0.00

Table 1: Comparación q_f teórico vs. $q_{f\text{-est}}$ reportado por el robot.

Gazebo ejecuta las configuraciones de forma determinista en bucle abierto. En un robot real, se esperarían errores significativos de odometría acumulada, especialmente en trayectos largos (4.6–5.9 m en estos escenarios).

La diferencia de posición y angular de 0 en todos los escenarios indica que el control de movimiento implementado en ROS2 ejecutó fielmente la secuencia de configuraciones generada por el planificador, sin acumulación de error apreciable en el simulador.

1.5 Posibles mejoras del método de planificación

- **Optimización de resolución de cuadrícula:** La resolución actual de 0.20 m fue aumentada (reducida desde 0.30 m) para garantizar viabilidad en escenarios con obstáculos complejos (especialmente Escenario 4). Un método adaptativo que ajuste dinámicamente la resolución según la densidad local de obstáculos podría balancear mejor entre precisión y tiempo de cómputo.
- **Suavizado de trayectoria:** Dijkstra sobre cuadrícula produce caminos con cambios bruscos de dirección (especialmente en movimientos diagonales). Se podría aplicar un algoritmo de suavizado como Bézier o B-splines para obtener trayectorias más fluidas y naturales.
- **Mayor resolución de cuadrícula:** usar celdas más pequeñas (por ejemplo, 10 cm) permitiría encontrar pasos más estrechos entre obstáculos y aproximar mejor las coordenadas de q_f .
- **A* en lugar de Dijkstra:** incorporar una heurística admisible (distancia euclídea al destino) reduciría el número de celdas exploradas, mejorando el tiempo de cómputo en escenarios grandes.
- **Manejo de orientación en el C-Space:** actualmente la expansión de obstáculos asume el cuadrado inscrito de 30 cm independientemente de la orientación. Un C-Space 3D que considere θ permitiría rutas más ajustadas cuando el robot se desplaza alineado con sus ejes.
- **Campos de potencial como métrica de coste:** en lugar de penalizar celdas semi-

libres con un factor fijo, se podría usar la distancia al C-obstáculo más cercano para generar caminos que maximicen la distancia a los obstáculos.

1.6 Decisiones de diseño y trade-offs

Durante la implementación se efectuaron las siguientes decisiones pragmáticas:

- a) **CELL_SIZE = 0.20 m:** Aunque el robot ocupa $30\text{ cm} \times 30\text{ cm}$, la resolución más fina garantiza viabilidad en escenarios con obstáculos complejos (Escenario 4). *Trade-off:* mejor cobertura de caminos vs. mayor tiempo de cómputo (típicamente $< 100\text{ ms}$).
- b) **Relocalización aditiva:** El método de corrección LiDAR es lineal y rápido. *Trade-off:* simplicidad e implementación rápida vs. precisión subóptima en escenarios asimétricos.
- c) **Open-loop execution:** El robot ejecuta configuraciones sin retroalimentación de control. *Trade-off:* complejidad reducida vs. acumulación de error odométrico ($\approx 1.31\text{ m}$ en 5 m).

2. Método de Relocalización

2.1 Descripción del método

Al finalizar la ejecución del camino geométrico, el robot se encuentra en la configuración estimada $q_{f\text{-est}}$, que es la posición que el robot cree haber alcanzado según su odometría interna. Para determinar la configuración real q_{act} , se emplea el sensor LiDAR 360° del robot, que mide las distancias a los obstáculos en cuatro direcciones principales: frente, atrás, derecha e izquierda.

El método de relocalización aprovecha que en la configuración q_f el robot debe quedar con un obstáculo a d_{frente} metros al frente y con otro obstáculo a d_{derecha} metros a la derecha. Dado que el escenario es conocido, se puede inferir la posición real del robot a partir de estas dos mediciones.

El procedimiento implementado utiliza un **modelo simplificado**: con la lectura d_{frente} del LiDAR se estima el error en la coordenada Y (dirección de avance), y con d_{derecha} se estima el error en la coordenada X (dirección lateral). Las correcciones se aplican como deltas aditivos:

$$\Delta y = d_{\text{frente}} - d_{\text{frente, esperado}}, \quad \Delta x = -(d_{\text{derecha}} - d_{\text{derecha, esperado}}).$$

Este enfoque proporciona una corrección rápida pero aproximada; un método basado en la geometría conocida del escenario podría proporcionar precisión superior.

2.2 Análisis de errores de localización

De los resultados obtenidos en simulación se observa que la diferencia entre $q_{f\text{-est}}$ y q_{act} es exclusivamente posicional (diferencia angular $= 0^\circ$ en todos los casos), con diferencias de posición que oscilan entre 1.00 m y 1.80 m . Las principales causas de este error son:

- **Acumulación de error odométrico:** el robot acumula pequeños errores en cada paso de traslación y rotación durante la ejecución del camino. Estos errores se suman y producen una desviación final respecto a q_f teórico.
- **Resolución de la cuadrícula:** los waypoints generados por Dijkstra corresponden a centros de celdas de 20 cm, por lo que existe una imprecisión inicial de hasta 10 cm en la posición objetivo.
- **Deslizamiento de ruedas:** en la ejecución real o en simulación con fricción imperfecta, las ruedas pueden deslizarse levemente durante las rotaciones, introduciendo error angular que se propaga como error posicional en los movimientos siguientes.

La ausencia de error angular confirma que el control de rotación en el simulador es preciso. Sin embargo, los errores posicionales (≈ 1.31 m promedio) revelan que el método de relocalización actual, basado en correcciones aditivas simples a partir de las mediciones LiDAR, es una aproximación lineal que podría mejorar con métodos geométricos más sofisticados que consideren la topología del escenario.

2.3 Resultados – tabla $q_{f\text{-est}}$ vs q_{act}

La siguiente tabla muestra la comparación entre la configuración estimada por el robot (odometría) y la configuración real calculada a partir de las lecturas LiDAR del escenario.

Escena	$q_{f\text{-est}} = (x, y, \theta)$	$q_{\text{act}} = (x, y, \theta)$	Dif. posición (m)	Dif. angular (°)
Escena 1	(3.25, 4.25, 90°)	(3.15, 4.34, 90°)	1.221	0.00
Escena 2	(3.25, 4.25, 90°)	(3.17, 4.31, 90°)	1.000	0.00
Escena 3	(3.25, 4.25, 90°)	(3.14, 4.32, 90°)	1.253	0.00
Escena 4	(3.25, 4.25, 90°)	(3.14, 4.34, 90°)	1.421	0.00
Escena 5	(3.25, 4.25, 90°)	(3.35, 4.40, 90°)	1.803	0.00
Escena 6	(3.25, 4.25, 90°)	(3.28, 4.36, 90°)	1.140	0.00

Table 2: Comparación $q_{f\text{-est}}$ (odometría) vs. q_{act} (relocalización LiDAR).

Nota: Los errores posicionales de ≈ 1.31 m promedio son consistentes entre escenarios, sugiriendo origen sistemático en la acumulación odométrica durante trayectorias similares. La ausencia de error angular (0°) indica que el control rotacional es preciso. El método de relocalización actual aplica correcciones aditivas simples; se recomienda validar estos resultados con un método geométrico más completo que considere la topología del escenario conocido para mejorar la precisión de q_{act} .

Los errores de posición son consistentes entre escenarios (promedio ≈ 1.31 m), lo que sugiere que el origen del error es sistemático y proviene principalmente de la acumulación odométrica durante trayectorias de longitud similar (4.6–5.9 m en estos escenarios).

2.4 Limitaciones del método de relocalización actual

El método implementado asume un modelo lineal de corrección basado en mediciones independientes del frente y la derecha. En escenarios más complejos donde los obstáculos no están alineados con los ejes coordenados, o donde existen múltiples opciones de posición compatibles con las mediciones, este enfoque puede no converger a la solución geométrica óptima. Una mejora futura sería implementar un **filtro de Kalman extendido (EKF)** o un método de máxima verosimilitud que considere todas las mediciones LiDAR simultáneamente.

3. Videos Ilustrativos

Se incluyen tres videos de ejecución del camino geométrico solución, cubriendo escenarios de complejidad creciente:

Escena 2 – Escenario con 5 obstáculos (complejidad media):

<https://youtu.be/KJNl5uveKC0>

La solución navega desde $q_0 = (0.75, 0.75, 0)$ hasta $q_f = (3.25, 4.25, 90)$ con un obstáculo adicional en la zona central respecto a la Escena 1. Dijkstra encuentra un camino de 13 waypoints con una distancia total de 4.59 m rodeando el obstáculo central por la derecha.

Escena 3 – Escenario con 8 obstáculos (alta densidad):

<https://youtu.be/Cb-xV5Nk8Dw>

Con 8 obstáculos distribuidos en el área, el planificador genera un camino de 16 waypoints y 5.12 m, el más largo del conjunto. La solución es especialmente interesante porque el robot debe zigzaguear entre los obstáculos de la mitad inferior antes de poder avanzar hacia q_f .

Escena 5 – Punto de inicio distinto (q_0 en el lado derecho):

<https://youtu.be/Gv9fD1ukc8A>

Esta escena es representativa porque $q_0 = (3.25, 0.75, 180)$ ubica el robot en el lado derecho del escenario mirando hacia la izquierda, con una barrera de tres obstáculos alineados en la zona media izquierda. El planificador genera 15 waypoints y 4.95 m, encontrando un paso por encima de la barrera para llegar a q_f .